

Question 1

10 marks

In each of the following, state whether the statement is **true** or **false**. Write **T** (for true) or **F** (for false).

1. Integrative programming combines one or more applications of the same kind (e.g., the same programming language).
2. Design patterns are guidelines on how to solve common software problem.
3. Each instance of a class has its own value of an instance variable.
4. An abstract class consists of only abstract methods.
5. XML stores values as text.
6. An XML schema defines the structure of data in an XML document.
7. Distributed object architecture is a design pattern.
8. In socket programming, the input stream of the client connects to the input stream of the server.
9. RMI is a webservice architecture
10. There is low coupling when independent applications work together

Answer: 1. F 2. T 3. T 4. F 5. T 6. T 7. T 8. F 9. F 10. T

Question 2

20 marks

1. Briefly answer the following.
 - 1.1. “An interface forms a contract with the class that implements it”. Briefly explain. (2)
 - 1.2. A class **Employee** inherits another class **Person**. Use this to explain the concept of polymorphism. (2)

1.1. Answer: an interface is a contract because the implementation class is forced to implement all the abstract methods in the interface. ✓✓

1.2. Answer: polymorphism means multiple existence: an instance of a subclass can also exist as an instance of the superclass✓. In this case, an instance of **Employee** can exist and behave as both as **Employee** and also as a **Person**. ✓✓

2. The code below shows class **Y** inherits **X**. The code illustrates **method override**, explain why this is example is wrong. (3)

```
package q2b;

public class X {
    public int m(int i) {
        return i;
    }
}
```

```
package q2b;
public class Y extends X{
    private double m(int i) {
        return i*i;
    }
}
```

Answer: method override is when a subclass inherits a method and implements it differently✓. This example is incorrect because the access level ✓ and return type ✓ of **m** in **Y** is not the same as inherited from **X**.

3. **A** is an abstract class, **I** is an interface and **C** is a concrete class. **A** implements **I**, and **C** inherits **A**. In each of the following, indicate whether the statement is **valid** or **invalid**.

Explain your answer.

3.1. A obj1=new I(); (2)

3.2. I obj2=new A(); (2)

3.3.A obj3=new C(); (2)

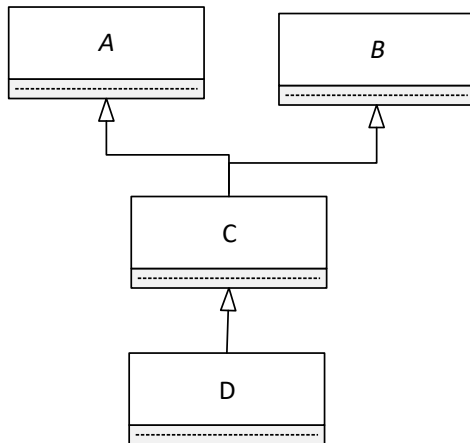
Answer:

3.1 A obj1=new I(); Invalid; cannot create an instance of an interface✓✓

3.2. I obj2=new A(); Invalid; cannot create an instance of an abstract class✓✓

3.3. A obj3=new C(); Valid; can assign an instance of C to a variable of A because C implements A✓✓

4. The UML diagram below shows the classes **A**, **B**, **C** and **D**. **A** and **B** are abstract.



4.1. Explain why you cannot write the Java code for the class **C**. (2)

4.2. If **A** contains only instance variables and instance methods, and **B** contains only static variables and abstract methods. Explain how you can resolve the problem. (3).

4.3. Write the class header for **C** after the problem has been resolved? (2)

Answer

4.1. C inherits two superclasses; Java does not support multiple inheritance. ✓✓

4.2. Maintain A as abstract class✓; convert B to an interface because it contains static variables and abstract methods✓; let C inherit only A and implement C✓

4.3. **public class C extends A implements B**✓✓

Question 3

25 marks

1. What are the main uses of the following design patterns? (4)

1.1. Singleton

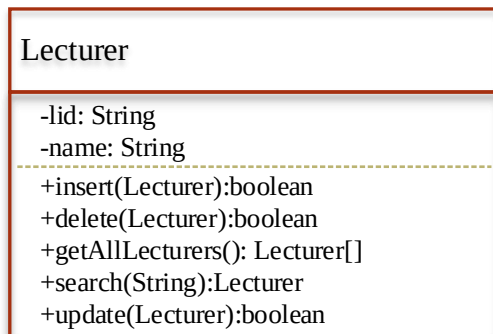
1.2. Factory

Answer

1.1. Singleton is used when only one instance of a class needs to be created✓✓

1.2. Factory is used to create instances of subclasses as instances of the superclass. ✓✓

2. The UML diagram below shows a class **Lecturer**. The methods **insert**, **delete**, **getAllLecturers**, **search** and **update** access **Lecturer** objects.

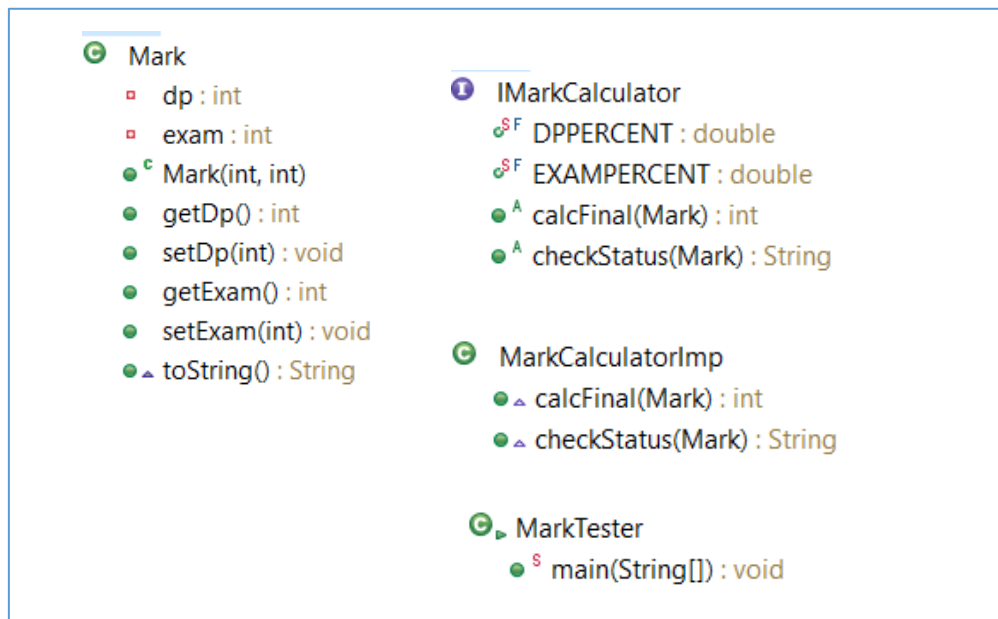


- 2.1. Briefly explain the problem with the class. (2)
 2.2. Which design pattern can resolve the problem? Briefly explain your answer. (2)

2.1. Answer: the Lecturer object is tightly tied to the methods that manipulate it. This can make the reduce code re-use). ✓✓

2.2. Answer: DAO; ✓ this separates the POCO from the methods that manipulate it. ✓ This way the POCO can be re-used in other code.

3. An application calculates the final mark in a module and determines the status (i.e., whether a student passed or failed). Below are the classes and interfaces in the application. **Mark** is a class. **IMarkCalculator** is an interface and **MarkCalculatorImp** implements **IMarkCalculator**. **MarkTester** creates instances of **Mark** and **MarkCalculatorImp**.



- **Mark** contains two data members (**dp** and **exam** marks) and the following methods: constructor (which initializes **dp** and **exam**), **get** and **set** (for **dp** and **exam**); and **toString** (which returns a string representation of **Mark**).
- **IMarkCalculator** contains two constant. **DPPERCENT** equals 40% and **EXAMPERCENT** equals 60%.
- **calcFinal** takes a **Mark** as a parameter and returns an integer (the final mark) calculated as 40% of the **dp** and 60% of the **exam** mark.
- **checkStatus** takes a **Mark** as a parameter and returns a string (“Failed” if the final mark is less than 50; otherwise, “Passed”).

3.1. Write the code for **IMarkCalculator**.

(5)

Answer

```
public interface IMarkCalculator {
    double DPPERCENT=0.4;
    double EXAMPERCENT=0.6;
    int calcFinal(Mark m);
    String checkStatus(Mark m);
}
```

3.2. Write the header of **MarkCalculatorImp**.

(2)

Answer

Answer: `public class MarkCalculatorImp implements IMarkCalculator`

3.3. Write the method **calcFinal** in **MarkCalculatorImp**.

(4)

Answer

```
public int calcFinal(Mark m) {
    return
    (int)Math.round((DPPERCENT*m.getDp()+EXAMPERCENT*m.getExam()));
}
```

3.4. Assuming all the other classes have been implemented, complete the main method in **MarkTester** as follows:

- 3.4.1. Create an instance (called **m**) of **Mark** with **dp** and **exam** values of 40 and 60 respectively. (1)
- 3.4.2. Create an instance (called **mci**) of **MarkCalculatorImp**. (1)
- 3.4.3. Call the appropriate method of **mci** with **m** to display the final mark. (2)
- 3.4.4. Call the appropriate method of **mci** with **m** to display the status (whether passed or failed). (2)

```

package q3;
public class MarkTester {
    public static void main(String[] args) {
        _____
        _____
        System.out.println("Final mark: " + ____);
        System.out.println("Status: " + _____);
    }
}

```

Answer

```

public class MarkTester {
    public static void main(String[] args) {
        Mark m=new Mark(40, 60); ✓
        MarkCalculatorImp mci=new MarkCalculatorImp();✓
        System.out.println("Final mark: " +mci.calcFinal(m)); ✓✓
        System.out.println("Status: " +mci.checkStatus(m)); ✓✓
    }
}

```

Question 4

25 marks

1. What is the main advantage of XML in Integrative programming? (2)

Any correct answer, e.g., XML allows disparate which applications interoperate to seamlessly share data✓✓

2. Bertha University of Technology offers several programs. Each program has many modules. Below is part of the data on modules: it shows the **code** and the **number** of registered students for a module.

IPRT 170
IEXP 250
PBDV 200

The data is stored as XML in a file called **module.xml**. The root node is **Program**. This contains many **Module** elements. Each **Module** has **code** (i.e., **code** of a module, an attribute and required) and **num** (i.e., **number** of registered students, an element).

2.1. Complete the XML code (below) in the file **module.xml** for the three modules. (5)

```
<?xml version="1.0" encoding="UTF-8"?>
```

Answer

```
<?xml version="1.0" encoding="UTF-8"?>
<Program>✓
  <Module code="IPRT">✓
    <num>170</num>✓
  </Module>
  ✓ <Module code="IEXP">
    <num>250</num>
  </Module>
  ✓ <Module code="PBDV">
    <num>200</num>
  </Module>
</Program>
```

2.2. Complete the **DTD** (below) for a file **module.dtd** which contains the DTD of the XML code in **module.xml**. (5)

```
<?xml version="1.0" encoding="UTF-8"?>
```

Answer:

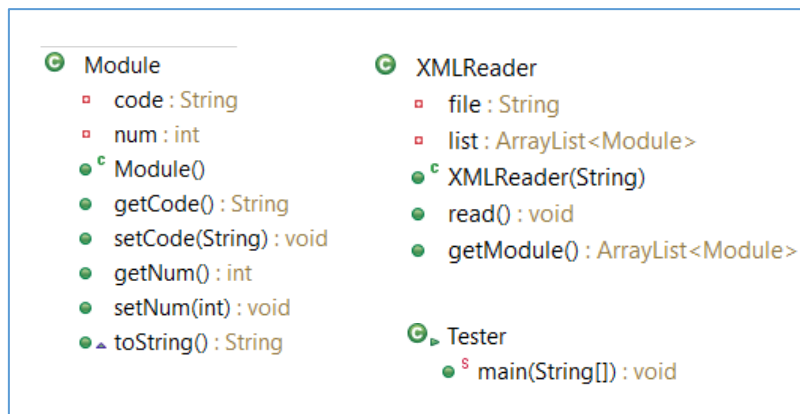
```
<?xml version='1.0' encoding='UTF-8'?>
<!ELEMENT Program (Module)*> ✓
<!ELEMENT Module (num)*> ✓
<!ATTLIST Module code CDATA #REQUIRED> ✓✓
<!ELEMENT num (#PCDATA)> ✓
```

2.3. Complete the line of code (below) in **module.xml** to link the XML to the DTD in **module.dtd**. (2)

```
<!DOCTYPE _____SYSTEM "_____">
```

Answer: `<!DOCTYPE Program SYSTEM "module.dtd">`

2.4. Below are the classes in an application that read the data in the XML file **module.xml**.



- Module** has two data members: **code** and **num** (i.e., the **code** of a module and the **number** of registered students); a **constructor**, **get**, **set** and **toString** methods.
- ReadXML** has one data member: **file** (i.e., the file to read from) and a static variable, **list** (a list to contain instances of **Module**).
 - The **constructor** takes a string (a file name) as a parameter to initialize **file**.
 - The method **read** uses **file** to create an instance (**xmlsr**) of **XMLStreamReader** to read the XML file to retrieve values of **code** and **num** to create instances of **Module** and store in **list**. At the end of the file, the method closes all streams.
 - The method **getModule** returns the **list** which contains instances of **Module**.
- Tester** creates an instance of **ReadXML** and calls:
 - The method **read**
 - The method **getModule** and displays the instance in **list**.

Below is part of the class **ReadXML** (**xmlsr** is an instance of **XMLStreamReader**).

Complete the following:

2.4.1. Write the condition of the loop, which tests if there are events in the file to read. (2)

2.4.2. Write the code such that if an event is a **start element** and the **name of the element** is "**Module**": (7)

- Create an instance of **Module**
- Get the value of the attribute **code** and
- Set the value of **code** of the instance.

2.4.3. Complete the method **getModule**, which returns the list of instances of **Module**. (2)

```
...
...
public class ReadXML {
    private String file;
    private static ArrayList<Module> list=new ArrayList<Module>();
    public ReadXML(String filePar) {
        file = filePar;
    }
    public void read() throws Exception{
        try {
            //Create stream
            FileReader fr=new FileReader(file);
            XMLInputFactory factory=XMLInputFactory.newInstance();
            XMLStreamReader xmlsr=factory.createXMLStreamReader(fr);

            int event;//declare a variable to store XML event
            String value=null;//declare a string to store characters
            Module m=null; //to assign new Module object

            while(_____){ //Complete for Question 2.4.1
                event=xmlsr.next();
                //Complete below for Question 2.4.2
                _____
                _____
                _____

            }
            //Close stream
            fr.close();
            xmlsr.close();
        }
        catch(Exception e) {
            System.out.println(e);
        }
    }
}
```

```

}
//Complete the code below for Question 2.4.3
public _____ getModule(){
    _____;
}
}

```

Answer

```

...
...

public void read(){
    try {
        //Create stream
        FileReader fr=new FileReader(file);
        XMLInputFactory factory=XMLInputFactory.newInstance();
        XMLStreamReader xmlsr=factory.createXMLStreamReader(fr);
        //Read XML data to create modules
        int event;//to store an XML event
        String value=null;//to store the characters in an element
        Module m=null; //to assign new Module object
        while(xmlsr.hasNext()) ✓✓{
            event=xmlsr.next();
            ✓if((event==XMLStreamConstants.START_ELEMENT)
&&(xmlsr.getLocalName().equals("Module"))✓✓✓) {
                m=new Module();✓
                m.setCode(xmlsr.getAttributeValue(0)) ✓✓;
            }
            else if(event==XMLStreamConstants.CHARACTERS) {
                value=xmlsr.getText();
            }
            else
if((event==XMLStreamConstants.END_ELEMENT)&&(xmlsr.getLocalName().equals("
num")))) {
                m.setNum(Integer.parseInt(value));
            }
            else
if((event==XMLStreamConstants.END_ELEMENT)&&(xmlsr.getLocalName().equals("
Module")))) {
                list.add(m);
            }
        }
        //Close stream
        fr.close();
        xmlsr.close();
    }
    catch(Exception e) {
        System.out.println(e);
    }
}
}

```

```

public ArrayList<Module>✓getModule(){
    return list;✓
}
}

```

Question 5

20 marks

1. Briefly explain why:

1.1. Exception handling is an important feature of distributed applications. (3)

Any correct answer: e.g., there are many things about distributed applications (e.g., sockets, streams, etc.) ✓ which can cause applications to stop abruptly (throw exceptions) ✓. Exception handling allows applications to avoid this when exception occurs. ✓

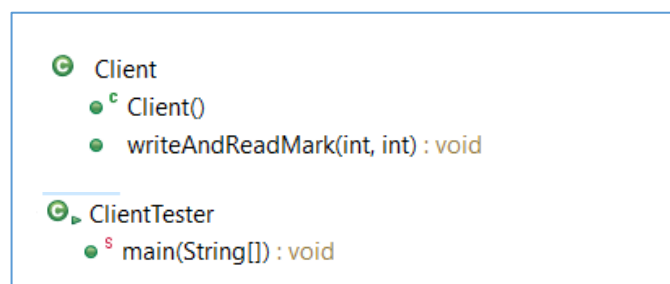
1.2. Distributed applications increase fault tolerance. (3)

Distributed applications consist of independent applications that work together✓. Distributed applications improve reliability (fault tolerance) ✓because other applications continue to perform their functions when others fail✓.

2. A client application writes two integers (**dp** and **exam** marks for a module) into a stream.

A server application reads the two numbers, calculates and writes the final mark for the module into the stream. The client reads the final mark and displays on the screen. The name of the server is “**localhost**” and the communication port is “**2025**”.

The client application consists of two classes: **Client** and **ClientTester**.



Client contains a method **writeAndReadMark**. This method takes two integer parameters (i.e., **dp** and **exam** marks) and:

a. Declares exception

- b. Creates a socket and appropriate streams
- c. Writes the two parameters to the stream
- d. Reads an integer (the final mark the server application writes) from the stream and displays the integer on the screen and
- e. Closes the streams and the socket

ClientTester contains a **main** method. This method:

- a. Declares exception
- b. Creates an instance (called **c**) of **Client** and
- c. Calls **writeAndReadMark** of **c** with the values 40 and 60

2.1. Complete the method **writeAndReadMark** in **Client**.

(10)

```

...
...
public class Client {
    public Client() {
    }

    public void writeAndReadMark(int dp, int exam)_____ {
        _____
        _____
        _____
        _____
        _____
        _____
        _____
        _____
        _____
        _____
    }
}

```

Answer:

```

...
...
package q5;

public class Client {

```

```
public Client() {  
}  
  
public void writeAndReadMark(int dp, int exam) throws Exception ✓{  
    //set up stream to write two numbers  
    Socket client=new Socket("localhost", 2025); ✓✓  
    OutputStream os =client.getOutputStream(); ✓  
    DataOutputStream dos=new DataOutputStream(os); ✓  
    //write two numbers  
    dos.writeInt(dp); dos.writeInt(exam); ✓  
  
    InputStream is = client.getInputStream(); ✓  
    DataInputStream dis = new DataInputStream(is); ✓.  
    System.out.println("Client here: " + dis.readInt()); ✓  
    client.close(); ✓  
}  
}
```

2.2 Complete the method **main** in **ClientTester**.

(4)

```
package q5;  
  
public class ClientTester {  
    public static void main(String[] args) _____{  
        _____  
        _____  
    }  
}
```

Answer:

```
public class ClientTester {  
  
    public static void main(String[] args) throws Exception ✓{  
        //call the client method to create socket, write, read and close socket  
        Client c=new Client(); ✓  
        c.writeAndReadMark(40, 60); ✓✓  
    }  
}
```